

TP4

Kinsley SUEDILE COLLET - B1 Info Ynov Montpellier

Partie 1

1. **Les adresses de a, b, c sont-elles consécutives ? Quelle est la différence entre elles ?**

Oui, elles se suivent en mémoire. La différence est de 4 octets d'écart entre chaque adresse, car un int occupe exactement 4 octets.

2. **Dans quel sens la stack grandit-elle sur votre machine ?**

La stack grandit vers le bas (des adresses les plus hautes vers les adresses les plus basses).

3. **Que devient la variable a une fois la fonction afficher terminée ?**

Elle est détruite automatiquement. Le bloc de la fonction est dépilé, donc la mémoire est libérée directement.

Partie 2

- **Programme A (Buffer overflow) :** La boucle va jusqu'à $i \leq 5$. Un tableau de taille 5 s'arrête à l'index 4. À l'indice 5, on écrit en dehors des limites.
Correction : Changer la condition de la boucle pour s'arrêter avant 5 (utiliser $i < 5$ au lieu de $\leq$).
- **Programme B (Dangling pointer) :** La fonction retourne l'adresse d'un tableau local (créé sur la stack). Quand la fonction se termine, le tableau est détruit. Le pointeur tape dans le vide.
Correction : Allouer le tableau dynamiquement sur le Heap avec malloc pour qu'il survive à la fin de la fonction.
- **Programme C (Buffer overflow sur chaîne) :** On alloue 5 octets, mais "Alexandre" + le `\0` de fin nécessitent 10 octets. Ça déborde.
Correction : Allouer suffisamment d'espace avec un malloc de 10 octets ($10 * \text{sizeof(char)}$).
- **2.3 Sanitizers :** Si on compile le Prog A avec `-fsanitize=address`, le terminal va afficher une erreur (généralement heap-buffer-overflow ou stack-buffer-overflow) en indiquant la ligne exacte où on a dépassé la taille du tableau alloué.

Partie 5

Concept	Définition	Exemple
Stack	Mémoire locale, rapide, se gère toute seule. Taille limitée.	<code>int a = 10;</code>
Heap	Mémoire dynamique, à gérer à la main. Énorme (limite = RAM).	<code>malloc()</code> ou <code>new</code>
malloc	Demande de la mémoire au système sur le Heap.	<code>malloc(10 * sizeof(int));</code>
free	Rend la mémoire au système. Toujours après un <code>malloc</code> !	<code>free(p);</code>
Memory leak	Fuite : faire un <code>malloc</code> mais oublier de faire le <code>free</code> à la fin.	Oublier <code>free(tab);</code>
Dangling pointer	Pointeur fou : pointe vers une zone mémoire déjà libérée.	Utiliser <code>p</code> après un <code>free(p)</code>
Buffer overflow	Déborder : écrire ou lire en dehors des limites d'un tableau.	<code>tab[5]</code> sur un array de taille 5
Classe	Plan de construction en	<code>class Etudiant {...};</code>

Concept	Définition	Exemple
	POO (regroupe variables et fonctions).	
Constructeur	Fonction spéciale appelée à la création pour initialiser l'objet.	Etudiant() { age = 0; }
Encapsulation	Cacher les variables internes pour protéger les données.	Mettre les attributs en private